

Reviewer Pack — Free Logarithmic Energy Deficit Lower-Bounds $R(\text{DISJ})$

Entry ea69d0cf842b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 03:37:36 UTC

Free Logarithmic Energy Deficit Lower-Bounds $R(\text{DISJ})$

Entry ID: ea69d0cf842b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 03:37:36 UTC

1. Conjecture

Field A (mathematical branch): Free probability — Voiculescu free logarithmic energy $\int f \log |x-y|^{-1} d\mu \otimes d\mu$ of empirical singular-value measures (microstate-free entropy χ^* , NOT free cumulants) **Field B** (complexity object): Randomized two-party communication complexity $R_{1/3}(f)$ of sign matrices $M_f \in \{-1, +1\}^{\{2^n \times 2^n\}}$, with DISJOINTNESS as the canonical $\Omega(n)$ target and discrepancy $\text{disc}(M_f)$ as the computable proxy via $R_{1/3}(f) \geq (1/2) \cdot \log_2(1/\text{disc}(M_f)) - O(1)$

Statement:

Let $M_f \in \{-1, +1\}^{\{N \times N\}}$ with $N=2^n$ be the sign communication matrix of a total Boolean function f , let $\sigma_1 \geq \dots \geq \sigma_N$ be its singular values, and let $\mu_f := (1/N) \sum_i \delta_{\{\sigma_i/\sqrt{N}\}}$ be the normalized empirical measure on $[0, \sqrt{N}]$. Define the free logarithmic energy deficit $\Delta E(M_f) := \frac{1}{2} \cdot \log(\sigma_1^2/N) - (2/(N(N-1))) \sum_{\{i < j\}} \log |\sigma_i/\sqrt{N} - \sigma_j/\sqrt{N}|$. We conjecture (i) for every total f , $\log_2(1/\text{disc}(M_f)) \geq (1/16) \cdot n \cdot \Delta E(M_f) - 2$, and (ii) $\Delta E(M_{\text{DISJ}_n}) \geq \frac{1}{4} \cdot \log((1+\sqrt{5})/(\sqrt{5}-1)) - o(1)$, so part (i) yields $R_{1/3}(\text{DISJ}_n) = \Omega(n)$ tightly via Lean-provable tensor-product spectral arithmetic of the 2×2 atom $[[1,1],[1,0]]$.

Rationale (proposer's reasoning):

DISJ's communication matrix factors as an n -fold Kronecker power of a fixed 2×2 atom whose two singular values are golden-ratio reciprocals, so its empirical spectrum lives on a multiplicative cascade with anomalously high logarithmic repulsion; free-probabilistic log-energy is exactly the right invariant to detect such cascade-spread spectra and is invisible to single-moment bounds (Frobenius, operator-norm) used in classical discrepancy proofs. The deficit ΔE penalizes both spectral concentration (large σ_1) and clustering (small pairwise log-distance), aligning with Forster-type sign-rank arguments without requiring sign-rank itself. Free log-energy avoids the natural-proofs barrier because it is not closed under random restrictions and not large-monotone — random sign matrices have $\Delta E \approx 0$ by Marchenko–Pastur log-energy saturation, so the invariant is sparse on the average function class.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3eb3fd6c80becfb2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{2..8\} \times 30$ seeds, conjecture is SUPPORTED iff: (A) for all families, no instance has $n \cdot \Delta E / 16 > \log_2(1/\text{disc}) + 3$; (B) cross-family regression of $\log_2(1/\text{disc})$ on $n \cdot \Delta E / 16$ yields slope ≥ 1.0 and $R^2 \geq 0.7$; (C) mean $\Delta E(\text{DISJ}_n)/n$ over seeds is ≥ 0.12 for $n \in \{6, 7, 8\}$ and monotone non-decreasing in n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - free logarithmic energy singular values communication complexity discrepancy - Voiculescu free entropy chi* sign matrix discrepancy lower bound - logarithmic potential empirical singular value measure disjointness randomized communication

Top relevant hits considered: - [http://arxiv.org/abs/1211.0310v1] Large Synoptic Survey Telescope: Dark Energy Science Collaboration - [http://arxiv.org/abs/astro-ph/0510346v1] The Dark Energy Survey - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1201.0716v2] Concavification of free entropy - [http://arxiv.org/abs/1412.0452v3] Entropy discrepancy and total derivatives in trace anomaly - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2207.14810v3] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [http://arxiv.org/abs/2102.12789v1] Quantum tunneling of a singular potential

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def sign_communication_matrix(f):
    n = len(f)
    M_f = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            M_f[i][j] = (-1) ** (f(i) ^ f(j))
    return M_f
```

```

def svd(M):
    # Simple power iteration method to approximate the largest singular value
    n = len(M)
    v = [random.random() for _ in range(n)]
    for _ in range(100): # Number of iterations
        v = [sum(M[i][j] * v[j] for j in range(n)) for i in range(n)]
        norm = sum(x**2 for x in v)
        v = [x / math.sqrt(norm) for x in v]
    sigma = max(abs(sum(M[i][j] * v[j] for j in range(n))) for i in range(n))
    return sigma

def log_energy_deficit(sigma_values):
    n = len(sigma_values)
    signal = max(sigma_values)
    sigma_values.remove(signal)
    sum_log_diff = sum(math.log(abs(sigma_i / math.sqrt(n) - sigma_j / math.sqrt(n))) for i in range(n-1))
    return 0.5 * math.log(signal**2 / n) - (2 / (n * (n - 1))) * sum_log_diff

def disc(M):
    # Top singular value bound
    signal = svd(M)
    return signal / len(M)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def DISJ(n, i):
        return lambda x: (x >> i) & 1

    n_values = [2, 3, 4, 5, 6, 7, 8]
    results = defaultdict(list)

    for n in n_values:
        f = DISJ(n, random.randint(0, n-1))
        M_f = sign_communication_matrix(f)
        sigma_values = sorted([svd(M) for M in [M_f]], reverse=True)
        delta_E = log_energy_deficit(sigma_values)
        disc_value = disc(M_f)

        results[n].append({
            "n": n,
            "delta_E": delta_E,
            "disc_value": disc_value
        })

    mean_delta_E_DISJ = sum(results[6]["delta_E"] for _ in range(30)) / 30
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for result in results[n]:
            if result["n"] * result["delta_E"] / 16 > math.log2(1 / result["disc_value"]) + 3:
                conjecture_holds = False
                counterexample = f"DISJ_{result['n']}"

```

```

    return {
        "metric_name": "log2(1/disc) vs n*DeltaE/16",
        "metric_value": mean_delta_E_DISJ,
        "instances_tested": len(results[6]),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_delta_E_DISJ = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_delta_E_DISJ} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"DISJ\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0f26f5f.py", line 94, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0f26f5f.py", line 59, in run_trial
    M_f = sign_communication_matrix(f)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0f26f5f.py", line 18, in sign_communication_matrix
    n = len(f)
        ~~~~
TypeError: object of type 'function' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`len()` called on a function object) before producing any data, so neither support nor falsification criteria could be evaluated. | next: Fix `sign_communication_matrix` to derive `n` from the function's domain (e.g., pass `n` explicitly or enumerate inputs in $\{0,1\}^n$) rather than calling `len(f)`, then rerun the $n \in \{2..8\} \times 30$ protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	141227
2	preregistration	claude_max	opus	0	0	7520
3	novelty	claude_max	opus	0	0	3482
4	novelty	claude_max	opus	0	0	9947
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27941
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31576
7	judge	claude_max	opus	0	0	4875

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 226569 ms total latency. Provider mix: `{'claude_max': 5, 'ollama_remote': 2}`

(full prompt+response transcripts available in `research/audit/ea69d0cf842b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ea69d0cf842b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ea69d0cf842b.tar.gz` (if generated)